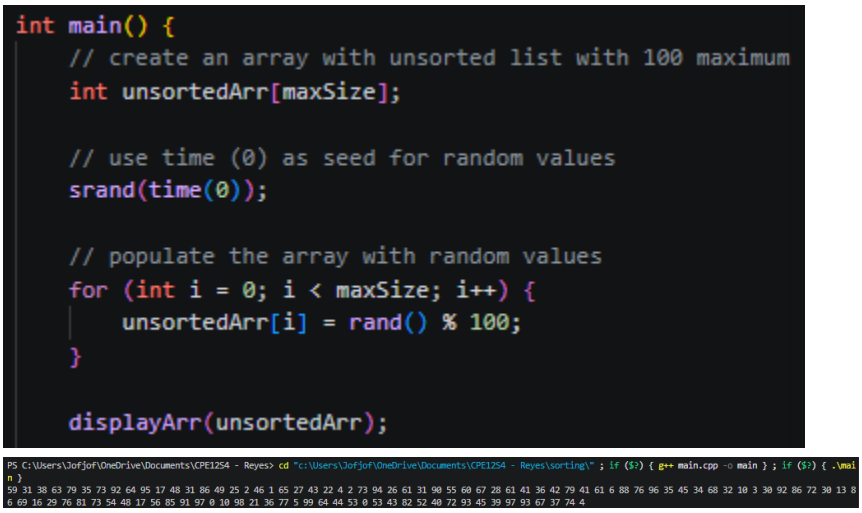
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/4/26	**DATE SUBMITTED:**	9/4/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 **Sorting Algorithim Pt1**

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	To begin the program, an array of 100 integers is created and then filled in each position with a random number provided by the rand() function. The output indicates that all items are placed in arbitrary order, but they must be arranged in proper order to ensure that everything is filled correctly before the sorting process begins.

```
int main() {
    // create an array with unsorted list with 100 maximum
    int unsortedArr[maxSize];

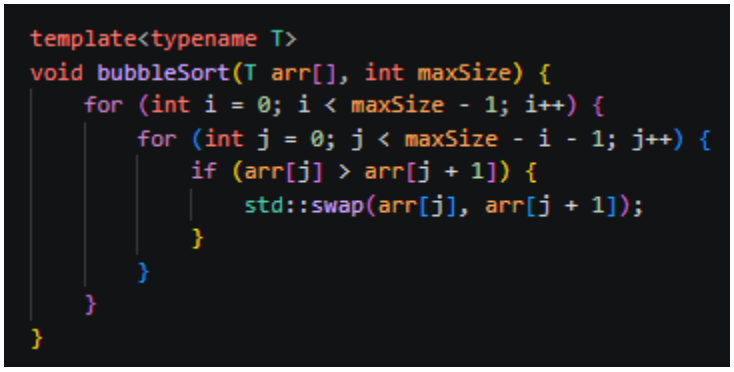
    // use time (0) as seed for random values
    srand(time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 100;
    }

    displayArr(unsortedArr);
```

```
PS C:\Users\JofJof\OneDrive\Documents\CPE12S4 - Reyes> cd "c:\Users\JofJof\OneDrive\Documents\CPE12S4 - Reyes\sorting\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\mai
n }
59 31 38 63 79 35 73 92 64 95 17 48 31 86 49 25 2 46 1 65 27 43 22 4 2 73 94 26 61 31 90 55 60 67 28 61 41 36 42 79 41 61 6 88 76 96 35 45 34 68 32 10 3 30 92 86 72 30 13 8
6 69 16 29 76 81 73 54 48 17 56 85 91 97 0 10 98 21 36 77 5 99 64 44 53 0 53 43 82 52 40 72 93 45 39 97 93 67 37 74 4
```

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
template<typename T>
void bubbleSort(T arr[], int maxSize) {
    for (int i = 0; i < maxSize - 1; i++) {
        for (int j = 0; j < maxSize - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

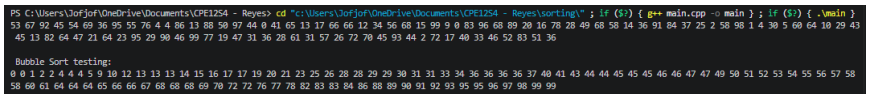
	
Observations	Initially, the program creates an array of up to 100 numbers and then uses random integers generated by the rand() function to fill in each position. The output displays the items in a random, odd order, arranged in clear segments to verify that the array is filled with the appropriate data before the sorting process begins.

Table 7-2. Bubble Sort Technique

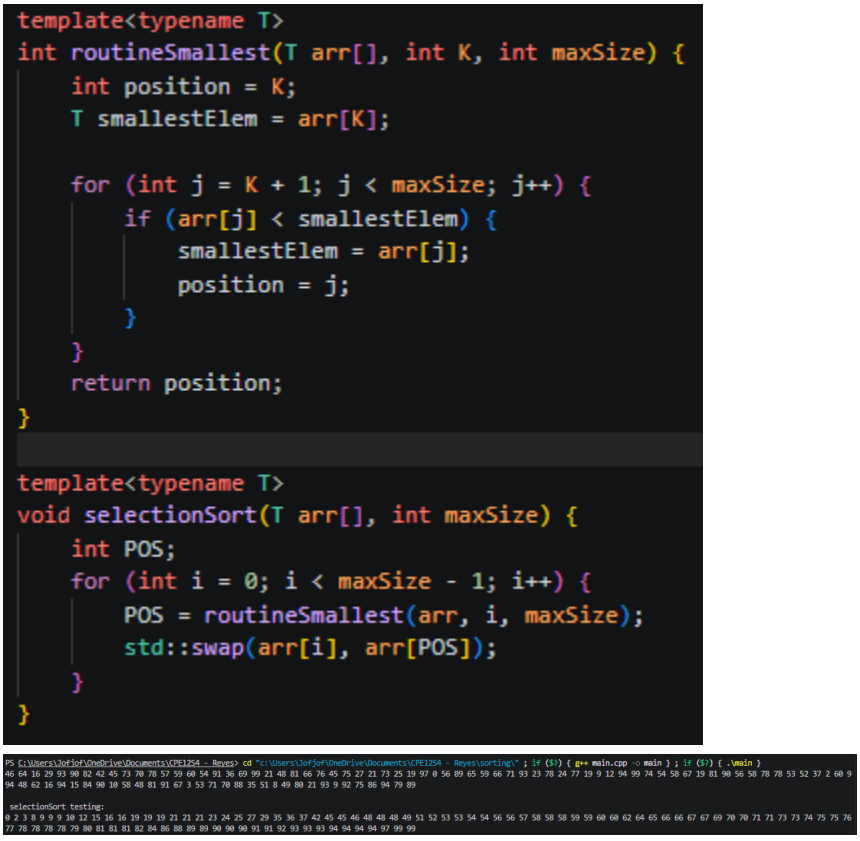
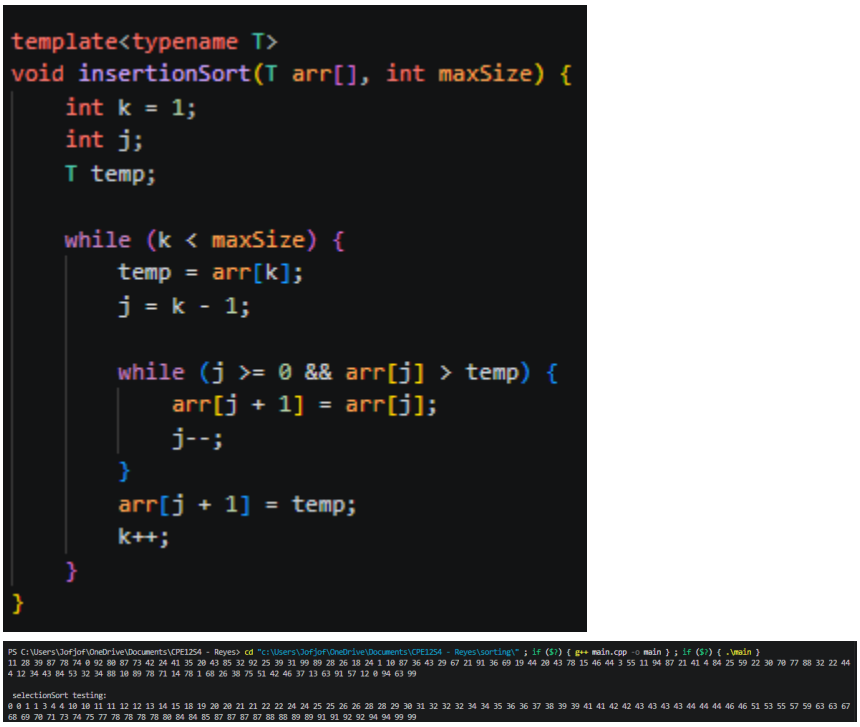
Code + Console Screenshot	
Observations	Selection Sort employs a helper algorithm called routineSmallest to search for the smallest value in an unsorted array. It then performs one swap per pass, returning the lowest value to the correct position at index i, and displays console results that the algorithm has followed the exact $N - 1$ steps from shortest to largest.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observations	Insertion Sort goes through each element one by one starting from the second element up to the 99th.It takes the current element and saves it in a temporary spot.Then, it moves all the bigger elements before it one place to the right so there's space for the current element in the right position. The last output on the console shows an array that is completely sorted, from the smallest number to the largest.

```
template<typename T>
void insertionSort(T arr[], int maxSize) {
    int k = 1;
    int j;
    T temp;

    while (k < maxSize) {
        temp = arr[k];
        j = k - 1;

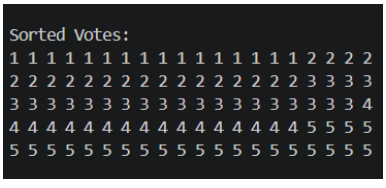
        while (j >= 0 && arr[j] > temp) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

```
PS C:\Users\JofJof\OneDrive\Documents\CPE1254 - Reyes> cd "c:\Users\JofJof\OneDrive\Documents\CPE1254 - Reyes\sorting\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
11 28 39 87 78 74 0 92 80 87 73 42 24 41 35 20 43 85 32 92 25 39 31 99 89 28 26 18 24 1 10 87 36 43 29 67 21 91 36 69 19 44 20 43 78 15 46 44 3 55 11 94 87 21 41 4 84 25 59 22 30 70 77 88 32 22 44
4 12 34 43 84 53 32 34 88 10 89 78 71 14 78 1 68 26 38 75 51 42 46 37 13 63 91 57 12 0 94 63 99

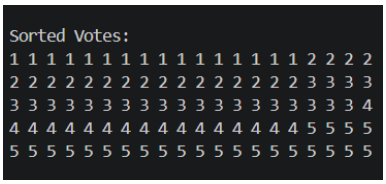
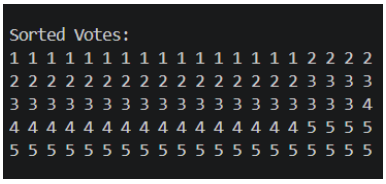
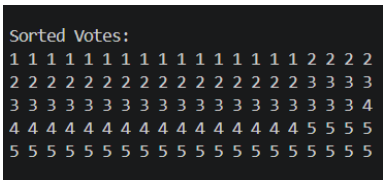
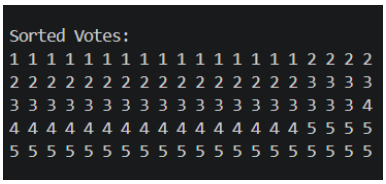
 selectionSort testing:
0 0 1 3 3 4 4 10 10 11 11 12 12 13 14 15 18 19 20 20 21 21 22 22 24 24 25 25 26 26 28 28 29 30 31 32 32 32 34 34 35 36 36 37 38 39 39 41 41 42 42 43 43 43 43 44 44 44 46 46 51 53 55 57 59 63 63 67
68 69 70 71 73 74 75 77 78 78 78 78 80 84 84 85 87 87 87 87 88 88 89 89 91 91 92 92 94 94 99 99
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	16 Votes	Candidate 1 (Bo Dalton Capistrano): 16 votes

```
Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

	20 Votes	Candidate 2 (Cornelius Raymon Agustin): 20 votes
	23 Votes	Candidate 3 (Deja Jayla Bañaga): 23 votes
	17 Votes	Candidate 4 (Laila Brielle Yabut): 17 votes
	24 Votes	Candidate 5 (Franklin Relano Castro): 24 votes

Question: Was your developed vote counting algorithm effective? Why or why not?

- **Yes, the vote-counting algorithm was effective because Insertion Sort successfully grouped the 100 randomized votes into contiguous blocks of candidate IDs, ensuring an accurate tally and correct winner identification with zero data loss.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini, QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, debugging runtime errors, paraphrasing text, formatting pseudocode, and drafting lab report responses.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used as a supportive tool to identify array out-of-bounds boundary errors (j >= 0 check), rephrase text for clarity, and structure pseudocode. All code execution,**

output testing, manual vote counting verification, and final analytical evaluations represent my own original work and critical thinking.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.